# claude-windows / 9b717b4b-673e-43c5-b3f4-33099e782c45

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9b717b4b-673e-43c5-b3f4-33099e782c45\subagents\workflows\wf_f26c1575-88a\agent-ad20cea4385b4ff99.jsonl`
- SHA-256: `3812209c5de9634b21d15a281878c043ee827a1c2d1aa498c56fa33e07b44cfe`
- Source modified: `2026-06-09T04:44:32+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_f26c1575-88a`
- session_id: `9b717b4b-673e-43c5-b3f4-33099e782c45`

## Transcript

### 1. user (2026-06-09T04:41:12.483Z)

Adversarially verify this claimed VLC 3.0.x Lua EXTENSION playback API against the actual VLC source (modules/lua/libs/playlist.c, variables.c, and how modules/lua/extension.c / vlc.c register libraries for the EXTENSION script type). Use WebFetch on the raw source files.

CLAIMS:

```
{
  "feasible": "yes",
  "availableInExtensions": "yes",
  "playback": "CONFIRMED from VLC 3.0 source (modules/lua/libs/playlist.c, vlclua_playlist_reg[]):\n\n- Play / start: vlc.playlist.play() -> C vlclua_playlist_play -> playlist_Play(). Starts/(re)starts playback.\n- Pause: vlc.playlist.pause() -> C vlclua_playlist_pause -> playlist_TogglePause(). *** This is a TOGGLE, NOT pause-only. *** Calling it while playing pauses; calling it while paused RESUMES.\n- Stop: vlc.playlist.stop() -> playlist_Stop().\n- Other controls in the same reg table: next, prev, skip(n), gotoitem(id), current, clear, add, enqueue, repeat_/loop/random, sort, search, get, delete, move.\n\nRESUME-FROM-PAUSE (the subtle part): there is no dedicated resume() function. Two reliable ways:\n  (a) Toggle-safe: query first, only toggle when needed:\n      if vlc.playlist.status() == \"paused\" then vlc.playlist.pause() end   -- toggles paused->playing\n  (b) State-deterministic resume that does NOT depend on current state: set the input \"state\" var directly:\n      local input = vlc.object.input()\n      if input then vlc.var.set(input, \"state\", 2) end   -- 2 = PLAYING_S; 3 = PAUSE_S (input_state_e in vlc 3.x)\n  Using \"state\" avoids the toggle ambiguity entirely (set to 3 = pause, 2 = play), which is the safest pattern when you have explicit Pause and Resume buttons. To be 100% certain you pause (vs accidentally resume), gate the toggle on status(): if status()==\"playing\" then pause() end. NOTE: pause() requires an input; if stopped it is a no-op.\n\nNO play_pause/togglepause helper is exposed under vlc.playlist ? pause() IS the togglepause (it calls playlist_TogglePause internally).",
  "seek": "CONFIRMED available in extensions and works from a dialog button callback.\n\nMechanism: there is no vlc.playlist.seek(). You seek by setting a variable on the INPUT object via the variables lib (modules/lua/libs/variables.c -> vlc.var.set(object, varname, value); modules/lua/libs/objects.c -> vlc.object.input()).\n\nABSOLUTE seek by time (microseconds in VLC 3.x):\n  local input = vlc.object.input()\n  if input then vlc.var.set(input, \"time\", target_us) end   -- target_us is INTEGER MICROSECONDS\n\nRELATIVE +/- N seconds (read-modify-write; vlc.var.get(input,\"time\") returns microseconds):\n  local input = vlc.object.input()\n  if input then\n    local t = vlc.var.get(input, \"time\")   -- microseconds\n    vlc.var.set(input, \"time\", t + N*1000000)     -- jump +N seconds (use - for back)\n  end\n\nABSOLUTE seek by fraction (0.0 .. 1.0):\n  vlc.var.set(input, \"position\", 0.5)   -- 50% of the media\n\nUNITS: \"time\" is in MICROSECONDS in VLC 3.0.x (CLOCK_FREQ = 1e6; multiply seconds by 1,000,000). \"position\" is a float 0..1. Community VLC-3 extension guide and forum threads corroborate microseconds. Practical note: \"position\" is sometimes more robust than \"time\" on some demuxers; \"time\" is exact when you have the offset. Always nil-check vlc.object.input() (it is nil when nothing is playing/stopped). Historic caveat: trac #6527 was about setting time from an INPUT-EVENT handler locking up; setting time from a normal dialog-button callback is the supported path and is fine.",
  "statusQuery": "CONFIRMED. vlc.playlist.status() -> C vlclua_playlist_status
```

(modules/lua/libs/playlist.c) -> playlist_Status(); returns a Lua string:\n  PLAYLIST_STOPPED -> \"stopped\"\n  PLAYLIST_RUNNING -> \"playing\"\n  PLAYLIST_PAUSED  -> \"paused\"\n  (else) -> \"unknown\"\nAvailable in extensions (luaopen_playlist is registered in the extension Lua state). Use it to make Pause/Resume deterministic, e.g. only toggle pause() when status() is in the desired state. (verg.ca lists a slightly larger set of literals incl. \"started\"/\"stopping\" from newer trees; the VLC 3.0 source returns exactly the four above. You can also read input state via vlc.var.get(vlc.object.input(),\"state\") -> integer: 2=PLAYING_S, 3=PAUSE_S.)",
  "caveats": [
    "pause() is a HARD TOGGLE (playlist_TogglePause), not pause-only. With separate Pause and Resume buttons you MUST either gate on vlc.playlist.status() (if 'playing' then pause()) or set the input 'state' var directly (3=pause, 2=play) to avoid accidentally resuming/pausing the wrong way.",
    "There is NO dedicated resume() and NO separate togglepause/play_pause under vlc.playlist; pause() IS the toggle. play() (re)starts playback but is not the idiomatic resume-from-pause (it can restart the current item) - prefer toggling pause() from 'paused', or set state=2.",
    "Seeking is NOT a playlist call; it goes through vlc.var.set on vlc.object.input(). vlc.object.input() returns nil when nothing is playing - ALWAYS nil-check before get/set or the extension errors.",
    "Units: 'time' is MICROSECONDS in VLC 3.0.x (x*1e6 for seconds); 'position' is 0..1 float. Do not pass seconds to 'time'.",
    "trac #6527: setting time from an INPUT-EVENT-loop handler could lock up; doing it from a normal dialog-button callback (the use case here) is the supported, safe path.",
    "Extensions can ONLY refresh their dialog from within a callback (no async push from a background timer in pure extensions); add_callback for the event loop is deprecated in VLC 3. For pure button-driven play/pause/seek this is irrelevant, but live progress-bar updates would need an interface script, not an extension.",
    "Some online VLC Lua API docs are for old/newer trees; the function set above is verified against the videolan/vlc-3.0 source specifically. Object held-reference: vlc.object.input() returns a held object; in long-lived loops you'd release, but for a one-shot button callback it's fine."
  ],
  "citations": [
    "https://github.com/videolan/vlc-3.0/blob/master/modules/lua/extension.c (GetLuaState(): registers luaopen_input, luaopen_object, luaopen_playlist, luaopen_variables, luaopen_dialog, luaopen_osd, etc. into the EXTENSION Lua state -- proves playlist/var/object/input ARE exposed to extensions)",
    "https://github.com/videolan/vlc-3.0/blob/master/modules/lua/libs/playlist.c (vlclua_playlist_reg[]: pause->vlclua_playlist_pause->playlist_TogglePause; play->playlist_Play; stop->playlist_Stop; status->playlist_Status returning 'stopped'/'playing'/'paused'/'unknown')",
    "https://github.com/videolan/vlc-3.0/blob/master/modules/lua/libs/variables.c (vlclua_var_reg[]: get, set, ... ; vlc.var.set(object, varname, value) is type-aware via var_Type/vlclua_tovalue -> var_Set)",
    "https://github.com/videolan/vlc-3.0/blob/master/modules/lua/libs/objects.c (vlclua_object_reg[]: {\"input\", vlclua_get_input} -> vlc.object.input() returns current input thread or nil)",
    "https://github.com/nima64/VLC-3-lua-extension-guide (community VLC 3 extension guide: vlc.var.get/set(vlc.object.input(),'time',...) in MICROSECONDS; 'position' 0..1; 'time-offset')",
    "https://forum.videolan.org/viewtopic.php?t=94399 (forum: pausing and jumping to a position via lua; time in microseconds, position seek)",
    "https://trac.videolan.org/vlc/ticket/6527 (caveat: setting time from input-events handler could lock up - not the dialog-button path)",
    "https://vlc.verg.ca/m/playlist/ (VLC Lua docs: playlist play/pause/stop/status)",
    "https://vlc.verg.ca/m/dialog/ (dialog:add_button(text, func) registers a Lua function called on click -> the callback can call playback/seek functions)"
  ],
  "confidence": "high"
}

For each claim confirm it exists and is reachable from an extension: play/pause/resume/stop names; whether pause() toggles; the seek mechanism + units; status query; and ESPECIALLY whether vlc.playlist and the var/input seek path are registered for extensions (not only for interface scripts). Correct anything wrong. Cite exact source files/lines.

**2. assistant (2026-06-09T04:41:16.158Z)**

I'll verify these claims against the actual VLC 3.0 source. Let me start by loading the WebFetch tool and fetching the key source files.

**3. assistant (2026-06-09T04:41:23.699Z)**

Now let me fetch the actual VLC 3.0 source files. The claim cites `videolan/vlc-3.0` on the `master` branch. Let me fetch the raw source files directly. I'll start with the most critical ones in parallel.

**4. user (2026-06-09T04:41:27.304Z)**

# Playlist Module Analysis

## (1) vlclua_playlist_reg[] Table

```c
static const luaL_Reg vlclua_playlist_reg[] = {
    { "prev", vlclua_playlist_prev },
    { "next", vlclua_playlist_next },
    { "skip", vlclua_playlist_skip },
    { "play", vlclua_playlist_play },
    { "pause", vlclua_playlist_pause },
    { "stop", vlclua_playlist_stop },
    { "clear", vlclua_playlist_clear },
    { "repeat", vlclua_playlist_repeat },
    { "repeat_", vlclua_playlist_repeat },
    { "loop", vlclua_playlist_loop },
    { "random", vlclua_playlist_random },
    { "goto", vlclua_playlist_gotoitem },
    { "gotoitem", vlclua_playlist_gotoitem },
    { "add", vlclua_playlist_add },
    { "enqueue", vlclua_playlist_enqueue },
    { "get", vlclua_playlist_get },
    { "search", vlclua_playlist_search },
    { "current", vlclua_playlist_current },
    { "sort", vlclua_playlist_sort },
    { "status", vlclua_playlist_status },
    { "delete", vlclua_playlist_delete },
    { "move", vlclua_playlist_move },
    { NULL, NULL }
};
```

## (2) vlclua_playlist_pause Body

Calls `"playlist_TogglePause( p_playlist );"` ? not playlist_Pause.

## (3) vlclua_playlist_play Body

Calls `"playlist_Play( p_playlist );"`.

## (4) vlclua_playlist_stop Body

Calls `"playlist_Stop( p_playlist );"`.

## (5) vlclua_playlist_status Return Strings

- `PLAYLIST_STOPPED`: `"stopped"`
- `PLAYLIST_RUNNING`: `"playing"`
- `PLAYLIST_PAUSED`: `"paused"`

- Default: `"unknown"`

**5. user (2026-06-09T04:41:30.130Z)**

# Lua State Initialization in GetLuaState Function

The `GetLuaState()` function (lines 757-837) initializes the Lua state and registers the following libraries:

**Confirmed registrations:**

1. **luaopen_playlist**: `"luaopen_playlist( L );"`
2. **luaopen_variables**: `"luaopen_variables( L );"`
3. **luaopen_object**: `"luaopen_object( L );"`
4. **luaopen_input**: `"luaopen_input( L );"`
5. **luaopen_dialog**: `"luaopen_dialog( L, p_ext );"`
6. **luaopen_osd**: `"luaopen_osd( L );"`

**Additional libraries registered in the same function:**

- `"luaopen_config( L );"`
- `"luaopen_msg( L );"`
- `"vlclua_fd_init( L, &p_ext->p_sys->dtable )"`
- `"luaopen_sd_intf( L );"`
- `"luaopen_stream( L );"`
- `"luaopen_strings( L );"`
- `"luaopen_video( L );"`
- `"luaopen_vlm( L );"`
- `"luaopen_volume( L );"`
- `"luaopen_xml( L );"`
- `"luaopen_vlcio( L );"`
- `"luaopen_errno( L );"`
- `"luaopen_win( L );"` (Windows-only)

All requested modules are present in the initialization sequence.

**6. assistant (2026-06-09T04:41:36.731Z)**

Both core claims confirmed. Now let me verify the seek mechanism (variables.c, objects.c) and double-check the exact line numbers and the `vlclua_playlist_pause` body, plus confirm `vlc.object.input()` returns nil when nothing is playing. Let me also verify the time units claim by checking the actual line numbers in the raw source.

**7. user (2026-06-09T04:41:41.414Z)**

# vlclua_object_reg[] Table

Yes, the table is reproduced verbatim below:

```c
static const luaL_Reg vlclua_object_reg[] = {
    { "input", vlclua_get_input },
    { "playlist", vlclua_get_playlist },
    { "libvlc", vlclua_get_libvlc },
    { "find", vlclua_object_find },
    { "vout", vlclua_get_vout},
    { "aout", vlclua_get_aout},
    { NULL, NULL }
};
```

Yes, `{"input", vlclua_get_input}` is present as the first entry.

### vlclua_get_input() Body

The function does return the current input thread. When there is no input, it pushes nil:

```c
static int vlclua_get_input( lua_State *L )
{
    input_thread_t *p_input = vlclua_get_input_internal( L );
    if( p_input )
    {
        /* NOTE: p_input is already held by vlclua_get_input_internal() */
        vlclua_push_vlc_object( L, p_input );
    }
    else lua_pushnil( L );
    return 1;
}
```

The function returns 1 in both cases (either the pushed object or nil), with nil explicitly pushed when no input thread exists.

#### 8. user (2026-06-09T04:41:42.161Z)

# Analysis of vlclua_var_reg[] Table

### Table Contents (Verbatim)

```c
static const luaL_Reg vlclua_var_reg[] = {
    { "inherit", vlclua_var_inherit },
    { "get", vlclua_var_get },
    { "get_list", vlclua_var_get_list },
    { "set", vlclua_var_set },
    { "create", vlclua_var_create },
    { "trigger_callback", vlclua_trigger_callback },
    { "libvlc_command", vlclua_libvlc_command },
    { "inc_integer", vlclua_inc_integer },
    { "dec_integer", vlclua_dec_integer },
    { "count_choices", vlclua_countchoices },
    { "toggle_bool", vlclua_togglebool },
    { NULL, NULL }
};
```

Yes, the table contains entries for both "get" and "set".

### Function Bodies

**vlclua_var_get:** Retrieves a variable's type, calls the underlying VLC getter, and pushes the result onto the Lua stack. The code shows: `"int i_type = var_Type( *pp_obj, psz_var ); if( var_Get( *pp_obj, psz_var, &val ) != VLC_SUCCESS ) return 0;"`

**vlclua_var_set:** Obtains the variable's type, converts the Lua value to the appropriate C type, and executes the underlying setter: `"int i_type = var_Type( *pp_obj, psz_var ); vlclua_tovalue( L, i_type, &val ); int i_ret = var_Set( *pp_obj, psz_var, val );"`

Both functions implement name-based lookup with type-dispatching through the `vlclua_tovalue()` and `vlclua_pushvalue()` helper functions.

#### 9. assistant (2026-06-09T04:41:58.207Z)

All four source files confirm the core mechanism. Now let me verify the remaining specific claims: (1) that "time" is in microseconds in VLC 3.0 (the input "time" variable definition),

and (2) confirm the `pause()` body requires/grabs an input or is safe. Let me check the input core variable definitions and confirm the playlist_TogglePause behavior. Let me also verify the exact behavior by checking the input.c / input variable registration for the "time" and "position" and "state" variables.

#### 10. user (2026-06-09T04:42:02.608Z)

I cannot find a `CLOCK_FREQ` definition in the provided content. The header file `vlc_common.h` does not contain this `#define` line.

The file does define `mtime_t` as a 64-bit integer type for storing high-precision dates or time intervals measured in microseconds, but the specific `CLOCK_FREQ` constant you're asking about is not present in this excerpt.

#### 11. user (2026-06-09T04:42:03.400Z)

# Input Variables Creation Analysis

## "time" variable

**var_Create call:**
```c
var_Create( p_input, "time", VLC_VAR_INTEGER );
```
Type: `VLC_VAR_INTEGER`. No explicit unit comment appears in the creation line. However, context suggests CLOCK_FREQ scaling is used elsewhere (e.g., in `input_ControlVarNavigation`, title length is divided by `CLOCK_FREQ`).

## "position" variable

**var_Create call:**
```c
var_Create( p_input, "position",  VLC_VAR_FLOAT );
```
Type: `VLC_VAR_FLOAT`. The `PositionCallback` function confirms it represents a normalized value: "if( i_length > 0 && newval.f_float >= 0.f && newval.f_float <= 1.f )" indicates the range is 0 to 1.

## "state" variable

**var_Create call:**
```c
var_Create( p_input, "state", VLC_VAR_INTEGER );
```
Type: `VLC_VAR_INTEGER`. The `StateCallback` function accepts `PLAYING_S` and `PAUSE_S` as valid enum values: "if( newval.i_int == PLAYING_S || newval.i_int == PAUSE_S )".

#### 12. assistant (2026-06-09T04:42:09.579Z)

"time" is VLC_VAR_INTEGER (consistent with microseconds, not float seconds), "position" is VLC_VAR_FLOAT 0..1, "state" is VLC_VAR_INTEGER accepting PLAYING_S/PAUSE_S. Let me confirm CLOCK_FREQ value and the PLAYING_S/PAUSE_S enum integer values to validate the "2=PLAYING_S, 3=PAUSE_S" claim, plus verify the StateCallback to confirm setting "state" var works for play/pause.

#### 13. user (2026-06-09T04:42:09.863Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

#### 14. user (2026-06-09T04:42:13.666Z)

# Input State Enum Analysis

The full enum is:

```c
typedef enum input_state_e
{
    INIT_S = 0,
    OPENING_S,
    PLAYING_S,
    PAUSE_S,
    END_S,
    ERROR_S,
} input_state_e;
```

Since `INIT_S` is explicitly set to 0 and subsequent members increment by 1:

- **PLAYING_S** evaluates to **2**
- **PAUSE_S** evaluates to **3**

**15. assistant (2026-06-09T04:42:19.572Z)**

The enum confirms PLAYING_S=2, PAUSE_S=3 exactly as claimed. The vlc_tick.h path 404'd (that file is from VLC 4.x; in VLC 3.0 CLOCK_FREQ is defined in vlc_common.h or mtime.h). Let me confirm CLOCK_FREQ in the right VLC 3.0 location. Let me check vlc_common.h more specifically and the State callback behavior.

**16. user (2026-06-09T04:42:24.413Z)**

I searched the provided file for "CLOCK_FREQ" and found no matches. Additionally, there are no #define statements containing this term.

Regarding time-related documentation, the file contains this comment about the `mtime_t` type:

"Store a high precision date or time interval. The maximum precision is the microsecond, and a 64 bits integer is used to avoid overflows (maximum time interval is then 292271 years, which should be long enough for any video). Dates are stored as microseconds since a common date (usually the epoch)."

This indicates that the time unit used throughout this header is **microseconds** (1/1,000,000 of a second).

**17. assistant (2026-06-09T04:42:30.863Z)**

mtime_t is documented as microseconds. CLOCK_FREQ in VLC 3.0 lives in `include/vlc_tick.h`? No ? that 404'd. It's actually in `mtime.h` or defined elsewhere. Let me confirm where CLOCK_FREQ=1000000 is defined in vlc-3.0 to fully nail the units claim.

**18. user (2026-06-09T04:42:34.550Z)**

I cannot find a `#define` for `CLOCK_FREQ` in the provided content. The web page excerpt shows definitions for `LAST_MDATE` and `MSTRTIME_MAX_SIZE`, but does not include a `CLOCK_FREQ` definition or any comment describing `mtime_t` units.

The header file provided appears to be incomplete or truncated, as it references `mtime_t` in function declarations and the `date_t` structure but does not show where `mtime_t` itself is defined or documented.

**19. assistant (2026-06-09T04:42:39.368Z)**

Let me locate CLOCK_FREQ via GitHub code search to confirm its value and file in vlc-3.0.

**20. user (2026-06-09T04:42:42.626Z)**

#define CLOCK_FREQ INT64_C(1000000)

```
#define INTF_IDLE_SLEEP                 (CLOCK_FREQ/20)
#define INPUT_IDLE_SLEEP                (CLOCK_FREQ/10)
#define DEFAULT_PTS_DELAY               (3*CLOCK_FREQ/10)
#define SPU_MAX_PREPARE_TIME            (CLOCK_FREQ/2)
#define VOUT_OUTMEM_SLEEP               (CLOCK_FREQ/50)
---trying vlc_common.h grep via raw---
```

**21. assistant (2026-06-09T04:42:47.729Z)**

Confirmed: `CLOCK_FREQ INT64_C(1000000)` is defined in `include/vlc_config.h`, proving 1 second = 1,000,000 in the time base. The microseconds claim is fully verified.

Now let me verify the one remaining subtle point in the pause body ? whether `vlclua_playlist_pause` calls `playlist_TogglePause` (the toggle claim) and whether `play()` could restart the current item. Let me grab the exact bodies of those functions with line numbers to make citations precise.

**22. user (2026-06-09T04:42:49.618Z)**

```
78:static int vlclua_playlist_play( lua_State * L )
79-{
80-    playlist_t *p_playlist = vlclua_get_playlist_internal( L );
81-    playlist_Play( p_playlist );
82-    return 0;
83-}
84-
85:static int vlclua_playlist_pause( lua_State * L )
86-{
87-    playlist_t *p_playlist = vlclua_get_playlist_internal( L );
88-    playlist_TogglePause( p_playlist );
89-    return 0;
90-}
91-
92:static int vlclua_playlist_stop( lua_State * L )
93-{
94-    playlist_t *p_playlist = vlclua_get_playlist_internal( L );
95-    playlist_Stop( p_playlist );
96-    return 0;
97-}
98-
--
387:static int vlclua_playlist_status( lua_State *L )
388-{
389-    playlist_t *p_playlist = vlclua_get_playlist_internal( L );
390-    PL_LOCK;
391-    int status = playlist_Status( p_playlist );
392-    PL_UNLOCK;
393-    switch( status )
--
418:    { "play", vlclua_playlist_play },
419:    { "pause", vlclua_playlist_pause },
420:    { "stop", vlclua_playlist_stop },
421-    { "clear", vlclua_playlist_clear },
422-    { "repeat", vlclua_playlist_repeat }, // repeat is a reserved lua keyword...
423-    { "repeat_", vlclua_playlist_repeat }, // ... provide repeat_ too.
424-    { "loop", vlclua_playlist_loop },
425-    { "random", vlclua_playlist_random },
426-#if LUA_VERSION_NUM < 502
--
436:    { "status", vlclua_playlist_status },
437-    { "delete", vlclua_playlist_delete },
438-    { "move", vlclua_playlist_move },
439-    { NULL, NULL }
440-};
441-
```

```
442-void luaopen_playlist( lua_State *L )
```

**23. assistant (2026-06-09T04:43:02.602Z)**

Exact bodies and line numbers confirmed. Let me grab the status() switch body and the var.set body line numbers to finalize precise citations, plus confirm playlist_TogglePause is genuinely a toggle (not pause-only) by checking its definition.

**24. user (2026-06-09T04:43:04.915Z)**

```
=== playlist_status switch body ===
    switch( status )
    {
        case PLAYLIST_STOPPED:
            lua_pushliteral( L, "stopped" );
            break;
        case PLAYLIST_RUNNING:
            lua_pushliteral( L, "playing" );
            break;
        case PLAYLIST_PAUSED:
            lua_pushliteral( L, "paused" );
            break;
        default:
            lua_pushliteral( L, "unknown" );
            break;
    }
=== playlist_TogglePause def (control.c) ===
```

**25. assistant (2026-06-09T04:43:09.504Z)**

The grep for `playlist_TogglePause` definition didn't match in control.c (different signature/file). Let me find its definition to confirm it's a true toggle.

**26. user (2026-06-09T04:43:10.481Z)**

(Bash completed with no output)

**27. user (2026-06-09T04:43:15.611Z)**

```
=== search control.c for Pause ===
125:    case PLAYLIST_PAUSE:
141:void playlist_Control( playlist_t *p_playlist, int query, int locked, ... )
```

**28. assistant (2026-06-09T04:43:20.548Z)**

`playlist_TogglePause` is a macro/inline wrapper in `vlc_playlist.h` that calls `playlist_Control` with `PLAYLIST_PAUSE`. Let me confirm the `PLAYLIST_PAUSE` case logic in control.c (line 125) and the macro definition to verify it toggles.

**29. user (2026-06-09T04:43:22.212Z)**

```
=== control.c PLAYLIST_PAUSE case ===
            pl_priv(p_playlist)->request.p_item = get_current_status_item( p_playlist );
            pl_priv(p_playlist)->request.i_skip = 0;
        }
        else
        if( var_GetInteger( pl_priv(p_playlist)->p_input, "state" ) == PAUSE_S )
            var_SetInteger( pl_priv(p_playlist)->p_input, "state", PLAYING_S );
        else
            var_SetInteger( pl_priv(p_playlist)->p_input, "state", PAUSE_S );
        break;

    case PLAYLIST_SKIP:
        pl_priv(p_playlist)->request.p_node = get_current_status_node( p_playlist );
```

```
        pl_priv(p_playlist)->request.p_item = get_current_status_item( p_playlist );
        pl_priv(p_playlist)->request.i_skip = (int) va_arg( args, int );
        pl_priv(p_playlist)->request.b_request = true;
        break;

    case PLAYLIST_PAUSE:
        if( pl_priv(p_playlist)->p_input == NULL )
            break;
        var_SetInteger( pl_priv(p_playlist)->p_input, "state", PAUSE_S );
        break;

    case PLAYLIST_RESUME:
        if( pl_priv(p_playlist)->p_input == NULL )
            break;
        var_SetInteger( pl_priv(p_playlist)->p_input, "state", PLAYING_S );
        break;
    }
    vlc_cond_signal( &pl_priv(p_playlist)->signal );
    PL_UNLOCK_IF( !locked );
}

=== TogglePause macro in vlc_playlist.h ===
233-#define playlist_Play(p) playlist_Control(p,PLAYLIST_PLAY, pl_Unlocked )
234:#define playlist_TogglePause(p) \
235-        playlist_Control(p, PLAYLIST_TOGGLE_PAUSE, pl_Unlocked)
236-#define playlist_Stop(p) playlist_Control(p,PLAYLIST_STOP, pl_Unlocked )
```

**30. assistant (2026-06-09T04:43:30.638Z)**

This is an important finding. `playlist_TogglePause` maps to `PLAYLIST_TOGGLE_PAUSE`, and I can see the toggle logic (lines ~112-116): it reads the input "state" var, and if PAUSE_S sets PLAYING_S, else sets PAUSE_S. This confirms the toggle behavior.

Critically, I also see that control.c has **separate** `PLAYLIST_PAUSE` (sets PAUSE_S) and `PLAYLIST_RESUME` (sets PLAYING_S) cases ? meaning there are dedicated pause-only and resume-only controls at the C playlist level. However, the question is whether these are exposed to Lua. The Lua reg table only maps `pause` ? `playlist_TogglePause` (PLAYLIST_TOGGLE_PAUSE), NOT `playlist_Pause`/`playlist_Resume`. Let me confirm there's no `playlist_Pause`/`playlist_Resume` macro exposed and that Lua doesn't bind them ? verifying the claim "NO dedicated resume() under vlc.playlist."

The reg table I already captured (lines 415-440) shows only `play`, `pause`, `stop` ? no `resume`, no `togglepause`, no `play_pause`. That confirms the claim. Let me do a final confirmation that the full reg table I saw has no resume/togglepause entries by viewing lines 415-440 completely.

**31. user (2026-06-09T04:43:31.689Z)**

```
/*****************************************************************************
 *
 *****************************************************************************/
static const luaL_Reg vlclua_playlist_reg[] = {
    { "prev", vlclua_playlist_prev },
    { "next", vlclua_playlist_next },
    { "skip", vlclua_playlist_skip },
    { "play", vlclua_playlist_play },
    { "pause", vlclua_playlist_pause },
    { "stop", vlclua_playlist_stop },
    { "clear", vlclua_playlist_clear },
    { "repeat", vlclua_playlist_repeat }, // repeat is a reserved lua keyword...
    { "repeat_", vlclua_playlist_repeat }, // ... provide repeat_ too.
    { "loop", vlclua_playlist_loop },
    { "random", vlclua_playlist_random },
#if LUA_VERSION_NUM < 502
    { "goto", vlclua_playlist_gotoitem },
```

```
#endif
    { "gotoitem", vlclua_playlist_gotoitem },
    { "add", vlclua_playlist_add },
    { "enqueue", vlclua_playlist_enqueue },
    { "get", vlclua_playlist_get },
    { "search", vlclua_playlist_search },
    { "current", vlclua_playlist_current },
    { "sort", vlclua_playlist_sort },
    { "status", vlclua_playlist_status },
    { "delete", vlclua_playlist_delete },
    { "move", vlclua_playlist_move },
    { NULL, NULL }
};
```

**32. assistant (2026-06-09T04:44:02.471Z)**

I have full source verification of every claim. Let me note one minor discrepancy: the claim's reproduction of the reg table lists `current` as appearing once and is accurate, but the WebFetch summary earlier listed `current` after `search` ? both match the real table. Also the claim's caveat about toggle logic ("set 3=pause, 2=play directly via state var") is now actually validated by the C toggle implementation which itself reads/writes the "state" var with PAUSE_S/PLAYING_S.

One subtlety to flag in corrections: the claim says "play() (re)starts playback but is not the idiomatic resume" ? and notes `playlist_Play` could restart the current item. The control.c `PLAYLIST_PLAY` case would need checking, but this is a soft/advisory claim, not a hard API error. The hard API facts are all confirmed.

Let me compile the verdict.

**33. user (2026-06-09T04:44:18.123Z)**

Structured output provided successfully

**34. assistant (2026-06-09T04:44:32.204Z)**

Verdict: **confirmed**. Every load-bearing API claim checks out against the actual VLC 3.0 source.

Key verified facts:
- **Extension exposure (the crux):** `modules/lua/extension.c` `GetLuaState()` registers `luaopen_playlist`, `luaopen_variables`, `luaopen_object`, and `luaopen_input` into the **extension** Lua state ? so `vlc.playlist`, `vlc.var`, `vlc.object.input()` are all reachable from an extension, not interface-only.
- **play/pause/stop/status names + bodies** confirmed in `playlist.c` (L78-97, L387-440). `pause()` ? `playlist_TogglePause` is a genuine toggle (proven in `src/playlist/control.c`, which flips the input `state` var PAUSE_S?PLAYING_S). No `resume`/`togglepause`/`play_pause` entry exists in the reg table.
- **status()** returns exactly `"stopped"`/`"playing"`/`"paused"`/`"unknown"`.
- **Seek path** is `vlc.var.set(vlc.object.input(), "time"/"position"/"state", v)` ? `var_Set`; `vlc.object.input()` returns nil when nothing plays (so the nil-check caveat is real).
- **Units:** `time` is VLC_VAR_INTEGER microseconds (`CLOCK_FREQ = 1000000` in `vlc_config.h`), `position` is VLC_VAR_FLOAT 0..1, `state` integer with PLAYING_S=2 / PAUSE_S=3 (`vlc_input.h`). The deterministic resume-via-state-var trick uses the exact mechanism VLC itself uses internally.

Corrections are minor/advisory only: the `play()`-restarts-current-item assertion and the trac #6527 / event-loop-deprecation caveats were not source-proven in this pass (the first wasn't traced to `PLAYLIST_PLAY`; the latter two are forum/design claims outside the cited files). None affect the core API conclusions. Also flagged that citations point at the `vlc-3.0` fork's moving `master` rather than a pinned tag.